Mastering SQL: Essential Clauses

Structured Query Language (SQL) is the standard language for managing and manipulating databases. While basic `SELECT` statements retrieve data, mastering clauses like `ORDER BY`, `LIMIT`, `OFFSET`, and `GROUP BY` allows you to organize, paginate, and aggregate your results effectively to derive real business insights.

1. ORDER BY

The `ORDER BY` clause is used to sort the result-set in ascending or descending order. By default, it sorts records in ascending order (A-Z, 0-9). To sort the records in descending order, use the `DESC` keyword.

Syntax

```
SELECT column1, column2 FROM table_name ORDER BY column1 [ASC|DESC];
```

Example Scenario

Suppose we have an **Employees** table. To list all employees sorted by their salary from highest to lowest:

```
SELECT first_name, last_name, salary
FROM Employees
ORDER BY salary DESC;
```

2. LIMIT and OFFSET

The `LIMIT` clause is used to specify the maximum number of records to return. It is incredibly useful on large tables with thousands of records, allowing you to return only a small subset to improve performance.

The `OFFSET` clause is used alongside `LIMIT` to skip a specific number of rows before beginning to return the rows. This combination is the standard way to implement **pagination** in web applications.

Syntax

```
SELECT column1, column2 FROM table_name
LIMIT number_of_rows OFFSET number_to_skip;
```

Example Scenario

If we want to get the 3rd and 4th highest-paid employees (meaning we need to skip the top 2, and then limit the result to the next 2):

```
SELECT first_name, last_name, salary
FROM Employees
ORDER BY salary DESC
LIMIT 2 OFFSET 2;
```

> **Important Note:** Always use `ORDER BY` when using `LIMIT` and `OFFSET`. Without an explicit sort order, a relational database might return an unpredictable subset of rows, as rows have no inherent order.

3. GROUP BY

The `GROUP BY` statement groups rows that have the same values into summary rows. It is almost always used in conjunction with aggregate functions like `COUNT()`, `MAX()`, `MIN()`, `SUM()`, and `AVG()`.

Syntax

```
SELECT column_name(s), AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name(s);
```

Example Scenario

Let's calculate the average salary and total number of employees for each department:

```
SELECT department_id, COUNT(*) as employee_count, AVG(salary) as avg_salary
FROM Employees
GROUP BY department_id;
```

Putting It All Together

These clauses can be combined to form powerful analytical queries. The standard logical order of execution (and syntax placement) in a SQL statement is:

1. `FROM` & `JOIN` (Determine the base dataset)
2. `WHERE` (Filter rows)

3. `GROUP BY` (Group rows based on common values)
4. `HAVING` (Filter groups)
5. `SELECT` (Select columns and compute aggregates)
6. `ORDER BY` (Sort the final results)
7. `LIMIT` / `OFFSET` (Restrict the number of returned rows)

Advanced Combined Example

Goal: Find the top 3 departments with the highest average salary, considering only employees who were hired after the year 2020.

```
SELECT department_name, AVG(salary) as avg_salary
FROM Employees
JOIN Departments ON Employees.department_id = Departments.id
WHERE hire_date >= '2021-01-01'
GROUP BY department_name
ORDER BY avg_salary DESC
LIMIT 3;
```